

Automatisation et supervision : l'application tourne seule, et se surveille

Plus rien ne dépend de quelqu'un qui lance une commande. Les bougies arrivent toutes les 15 minutes, le bot tourne en continu, la dérive est mesurée chaque matin, un nouveau modèle est mis à l'épreuve chaque semaine, chaque modification du code est vérifiée avant d'être publiée, et des alertes signalent ce qui se dégrade.

Dépôt DataScientest-Studio/jul26_bde_crypto Date 23 septembre 2026

3 DAG AIRFLOW	15 min ENTRE DEUX COLLECTES	5 CONTRÔLES AVANT CHAQUE IMAGE	12 RÈGLES D'ALERTE
------------------	--------------------------------	-----------------------------------	-----------------------

01 En bref

TROIS BOUCLES, TROIS OUTILS

Une application de machine learning change pour trois raisons : son **code** évolue, ses **données** arrivent, son **modèle** vieillit. Chacune a sa boucle d'automatisation. Le code passe par l'intégration continue (GitHub Actions) ; les données et le modèle passent par Airflow ; la supervision (Prometheus et Grafana) regarde les trois.

Ce qui était demandé, et où le trouver

DEMANDE	RÉALISÉ	OÙ
Automatiser les étapes précédentes	OUI 3 DAG : collecte et bot, dérive, réentraînement	airflow/dags/
Pipeline d'intégration continue	OUI 5 contrôles, puis images publiées sur le registre GitHub	.github/workflows/ci.yml
Monitorer en production	OUI métriques, sondes, 12 alertes, tableau de bord	monitoring/, api/exportateur.py
Réentraînement du modèle	OUI hebdomadaire ou sur dérive, avec duel avant publication	scripts/reentrainer.py

```
python -m scripts.generer_secrets # une fois
docker compose up -d --build # 16 services, dont Airflow, MLflow, Prometheus, Grafana
python scripts/verifier_deploiement.py --profils pipeline supervision
```

SERVICE	ADRESSE	POUR QUOI FAIRE
Interface	localhost:8080	Le terminal : bougies en direct, décisions, carnet de trades
Airflow	localhost:8088	Les trois DAG, leurs exécutions et leurs journaux
MLflow	localhost:5000	Chaque réentraînement, et le registre des modèles
Grafana	localhost:3000	Le tableau de bord de production
Prometheus	localhost:9090	Les métriques brutes et l'état des alertes

02 Automatiser : Airflow

DAG	QUAND	CE QU'IL FAIT
cryptobot_collecte	toutes les 15 min	Bougies Binance → MongoDB (brut) → PostgreSQL (nettoyé), contrôle qualité, variables techniques, et un tour du bot pour chaque paire, pas de temps et style
cryptobot_derive	chaque matin, 6 h 30	Dérive de chaque paire et pas de temps ; déclenche le réentraînement si elle est forte
cryptobot_reentrainement	chaque dimanche, 3 h	Extrait figé, nouveau modèle, duel contre le modèle en service, publication seulement s'il gagne

Airflow orchestre, il ne calcule pas

Chaque tâche lance une commande du projet — `python -m scripts.ingestion_continue`, `python -m scripts.reentrainer entrainer`... — exactement celles qu'on lance à la main. Un DAG n'est qu'un calendrier, un ordre d'exécution et une politique de nouvelles tentatives. Deux bénéfices : la logique reste testable hors d'Airflow, et une panne se reproduit en recopiant une ligne de commande.

Ces commandes tournent dans **leur propre environnement Python**, installé dans l'image à côté de celui d'Airflow. Airflow fige les versions de centaines de bibliothèques ; scikit-learn et MLflow en demandent d'autres. Deux environnements, aucun conflit — et le modèle est entraîné avec exactement la version de scikit-learn que l'API utilise pour le charger.

Des choix de calendrier qui ont une raison

- **À 1, 16, 31 et 46 minutes, pas à la minute pile.** Une bougie de 15 minutes se clôture à 14 min 59 s. Lancer à l'heure pile, c'est risquer d'arriver avant et de la manquer jusqu'au passage suivant.
- **Pas de rattrapage des passages manqués.** L'ingestion repart toujours de la dernière bougie en base : un seul passage comble n'importe quel trou. Le jour du lancement, il a rattrapé 25 jours de retard — 15 700 bougies — en 15 secondes.
- **Une seule exécution à la fois** pour chaque DAG : deux collectes simultanées écriraient les mêmes bougies.

- **Trois nouvelles tentatives, espacées de plus en plus** : la plupart des échecs réels sont passagers (Binance qui répond une erreur, une base qui redémarre). Sauf pour le contrôle qualité : réessayer ne change pas un constat.

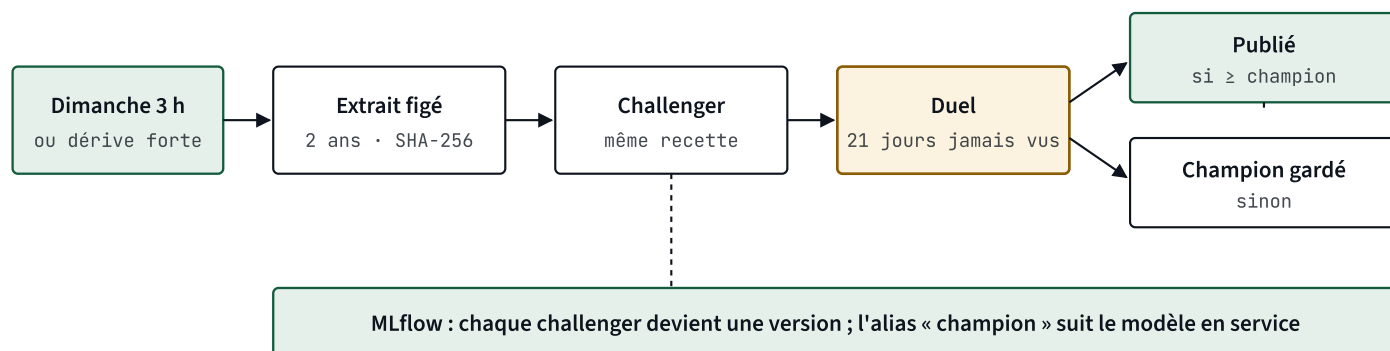
Le bot tourne ici, plus dans la page

À l'étape 4, le carnet de positions n'avancait que lorsqu'on ouvrait l'interface. Page fermée pendant trois jours, trois jours de signaux jamais vus. Désormais, Airflow appelle l'API toutes les 15 minutes pour chacune des 30 combinaisons — avec sa propre clé, comme n'importe quel client. Un tour complet prend 90 secondes.

Le contrôle qualité : des tests, pour les données

Un modèle nourri de bougies manquantes ou incohérentes produit des prédictions fausses *sans jamais lever d'erreur*. À chaque passage, trois vérifications par série : la dernière bougie est-elle récente, manque-t-il des bougies, y en a-t-il d'impossibles (un plus haut sous le plus bas) ? Chaque résultat est archivé, et une incohérence fait échouer la tâche, donc se voit.

03 Le réentraînement : champion contre challenger



Le modèle en service n'est remplacé qu'à la toute fin, et seulement si le nouveau fait au moins aussi bien sur une période qu'il n'a jamais vue.

Pourquoi un duel, plutôt que « le plus récent est le meilleur »

Un réentraînement peut rendre un modèle *moins bon* : période d'apprentissage atypique, données manquantes, bug introduit dans le code. Sans comparaison, la production se dégraderait en silence. Avec, le pire cas est « on garde le modèle actuel ».

ÉTAPE	RÈGLE	POURQUOI
Données	Un extrait figé des deux dernières années, signé par une empreinte SHA-256, vérifiée avant l'entraînement	On n'entraîne jamais sur des données vivantes : le résultat doit pouvoir être reproduit à l'identique
Recette	Celle du modèle final de l'étape 3 : forêt aléatoire, calibration, deux seuils par style	Le duel compare des données récentes, pas deux méthodes
Évaluation	Les 21 derniers jours , retirés de l'apprentissage, avec un jour d'écart	L'étiquette d'une bougie dépend de la suivante : sans cet écart, la fin de l'apprentissage « verrait » le début de l'évaluation
Promotion	Au moins 30 ordres, et une précision au moins égale à celle du champion (style conservateur)	En dessous de 30 ordres, la mesure est du bruit
Publication	Copie à côté puis renommage ; l'ancien modèle est archivé	L'API ne lit jamais un fichier à moitié écrit, et un retour en arrière reste possible

L'API voit que le fichier a changé et **recharge le modèle d'elle-même** à l'appel suivant, sans redémarrer. La référence de dérive est régénérée en même temps : elle doit toujours décrire les données du modèle *en service*.

Le premier duel, le 23 septembre

MODÈLE	DONNÉES JUSQU'AU	ORDRES	PRÉCISION	MARGE D'ERREUR (95 %)
Champion (étape 3)	24 août 2026	447	0,575	0,529 – 0,620
Challenger	1 ^{er} septembre 2026	539	0,558	0,516 – 0,600

Mesure commune : les ordres du style conservateur sur les 21 derniers jours, que ni l'un ni l'autre n'a vus. **Le challenger est rejeté** et le modèle en service reste. L'écart de 1,7 point est dans la marge d'erreur : les deux modèles se valent, et à valeur égale ou incertaine, la règle garde celui qui a déjà fait ses preuves. Le même duel, relancé trois fois dans la journée, a donné exactement les mêmes chiffres, à la quatrième décimale : c'est la reproductibilité que garantit l'extrait figé.

LA LEÇON LA PLUS UTILE DE CETTE ÉTAPE

Le même jour, la dérive est forte sur les 15 séries surveillées : le marché bouge avec une amplitude qu'il n'avait pas pendant l'apprentissage. Et pourtant, le modèle en service garde 57,5 % de bonnes réponses sur ces trois semaines, dans la marge des 60 % mesurés à l'étape 3. **Une dérive des données n'est pas, à elle seule, une baisse de performance.** C'est pourquoi la dérive déclenche un duel, et non une mise en service automatique.

La dérive déclenche bien un réentraînement anticipé, mais avec un garde-fou : pas plus d'un tous les trois jours. Une dérive forte ne disparaît pas en une nuit ; sans ce garde-fou, elle provoquerait un réentraînement chaque matin sur des données presque identiques.

04 L'intégration continue

À chaque envoi de code sur GitHub, cinq contrôles indépendants tournent en parallèle. Les images ne sont construites et publiées que si **tous** sont verts. Durée totale : 5 minutes.

CONTRÔLE	CE QU'IL ATTRAPE
Analyse statique	Les erreurs visibles sans exécuter : nom inconnu, import inutile, variable jamais lue (ruff)
Tests unitaires	Les 149 tests, avec la mesure de couverture du code
Intégrité des DAG	Chaque DAG se charge dans un vrai Airflow 3.3, dans l'ordre prévu. Un DAG qui ne se charge pas disparaît <i>silencieusement</i> de l'interface d'Airflow.
Configuration	Règles d'alerte Prometheus et configuration nginx valides
Déploiement de bout en bout	Toute l'application démarrée de zéro sur une machine vierge, avec des secrets neufs, puis vérifiée de l'extérieur

Les trois images — `api`, `interface`, `airflow` — sont alors publiées sur le registre de GitHub, étiquetées du commit. Mettre à jour une machine revient à `docker compose pull && docker compose up -d` : elle récupère exactement ce qui a été testé.

POURQUOI AIRFLOW NE FAIT PAS L'INTÉGRATION CONTINUE

Les deux mots se ressemblent, mais pas les problèmes. L'intégration continue réagit à un *changement de code* : elle doit tourner à chaque envoi, sur une machine propre, et bloquer une version défectueuse avant qu'elle n'existe. Airflow réagit au *temps qui passe* : des données arrivent, un modèle vieillit. On dit parfois que le réentraînement planifié est l'« entraînement continu » d'une application de ML : c'est ce rôle que tient Airflow ici.

L'intégration continue a prouvé son utilité dès le premier jour : elle a refusé deux versions. Airflow 3.3 avait changé la façon de charger les DAG, ce que la machine de développement ne montrait pas ; et une vérification dépendait d'un délai fixe qui passait par chance en local.

05 Superviser

OUTIL	RÔLE
Prometheus	Lit les métriques toutes les 15 secondes et évalue les règles d'alerte
API	Expose ses propres métriques : appels et durée par route, erreurs, décisions prises, refus de clé
Exportateur	Traduit l'état du pipeline, lu en base, en métriques : retard des données, dérive, dernier tour du bot, issue du dernier duel
Sondes	Appellent chaque page web comme le ferait un utilisateur : interface, API à travers le proxy, Airflow, MLflow
Grafana	Le tableau « CryptoBot – production », décrit en fichier et chargé au démarrage

Les questions que posent les alertes

QUESTION	ALERTE	NIVEAU
Le service répond-il ?	API injoignable, page en panne (sondes), plus de 5 % d'erreurs, réponses lentes	CRITIQUE
Les données arrivent-elles ?	Plus de bougie depuis 3 intervalles, contrôle qualité en échec, base illisible	CRITIQUE
Le bot tourne-t-il ?	Aucun tour de carnet depuis 45 minutes	CRITIQUE
Le modèle est-il encore adapté ?	Dérive forte généralisée, dérive non mesurée depuis 2 jours, aucun duel depuis 9 jours	AVERTISSEMENT
Quelqu'un force-t-il la porte ?	Plus de 30 appels par minute refusés pour clé invalide	AVERTISSEMENT

Chaque alerte doit **durer** avant de se déclencher (de 2 minutes à 1 heure selon le cas) : un redémarrage de 30 secondes ne doit réveiller personne. Le 23 septembre, une seule alerte était en attente : la dérive forte généralisée. Elle dit vrai.

Pourquoi un exportateur à part

Les questions les plus importantes en production ne concernent pas l'API : les données arrivent-elles, le bot tourne-t-il ? Les réponses sont en base. Un petit service, construit sur la même image que l'API, les lit et les traduit en métriques — ce qui permet à Prometheus d'*alerter*, là où Grafana seul ne ferait qu'afficher. Il tourne à part pour survivre à une panne de l'API, et met ses lectures en cache une minute pour ne jamais surcharger la base.

Grafana lit aussi PostgreSQL directement pour l'historique (trades, capital virtuel, duels, dérive), avec un compte en **lecture seule** : un tableau de bord n'a aucune raison de pouvoir effacer une donnée.

06 Ce que la mise en route a révélé

Faire tourner l'application en continu a mis au jour des défauts qu'aucun lancement manuel n'avait montrés. Chacun est corrigé, et la plupart sont désormais couverts par un test.

DÉFAUT	COMMENT IL S'EST VU	CORRECTION
Carnet de positions à trous	Aucun trade après le 18 septembre : le bot ne tournait que page ouverte	Rattrapage des bougies manquées, puis bot confié à Airflow
Index MongoDB	Deuxième collecte en échec : un index unique de l'étape 2 exigeait un numéro de lot	Numéro de lot = ouverture de la première bougie
Fenêtre de dérive	Le 1h et le 4h étaient mesurés sur 360 jours et 2 ans au lieu de 90 jours	Même durée pour chaque pas de temps ; rapport de l'étape 4 corrigé
Enregistrement dans MLflow	Le modèle refusé par MLflow : une bibliothèque plus récente dans l'image jugeait un type de scikit-learn non sûr	Type déclaré, version de la bibliothèque figée
Registre incomplet	L'alias « champion » ne désignait rien : le modèle de l'étape 3 n'y avait jamais été inscrit	Inscription du modèle en service au premier duel

07 Limites et suite

LIMITE CONNUE	CONSÉQUENCE	PISTE
Alertes sans notification	Elles se lisent dans Grafana et Prometheus, mais personne n'est prévenu	Ajouter Alertmanager, vers un courriel ou un canal de discussion
Une seule machine	Si elle s'arrête, tout s'arrête ; le carnet se rattrape au redémarrage	Héberger sur un serveur, avec une sauvegarde des volumes
Pas de HTTPS	Acceptable tant que tout reste sur 127.0.0.1	Terminaison TLS devant nginx avant toute ouverture au réseau
Secrets dans un fichier .env	Protégé par les droits du fichier seulement	Un coffre à secrets en production réelle
Le modèle n'est toujours pas rentable	L'automatisation entretient un modèle au bord du hasard, sans le rendre rentable	Changer de cible (mouvements assez grands pour couvrir les frais), pas seulement de données

Pipeline airflow/, scripts/reentrainer.py, scripts/ingestion_continue.py

CI .github/workflows/ci.yml Supervision monitoring/, api/exportateur.py